

Cleartext WebView Transport in a2_case1.apk Enabling On-Path Content Tampering

Rongbang Cheng
540506620

Sienna Cai
540907399

Yuzhi Shao
550139687

Letian Chen
541012940

Jingzhao Shi
540895498

Abstract

We analyzed a2_case1.apk, package com.example.mastg_test0019, for the intended Part A insecure transport issue. The app permits cleartext traffic and loads `http://www.example.com` in a launcher-reachable WebView. An on-path attacker can observe or modify WebView HTTP traffic before it is rendered in the app UI. The supported impact is WebView content integrity compromise, not account takeover or backend compromise. The direct remediation is to enforce HTTPS, disable cleartext transport, and reject TLS validation failures.

1 Scope and Method

We decompiled the APK and inspected the manifest plus `MainActivity`. The finding is scoped to one issue: cleartext WebView loading that enables MITM-style content tampering. We do not claim credential theft, session-token theft, personal-data exposure, server compromise, or completed runtime MITM capture because the evidence does not establish those stronger claims.

2 System and Threat Model

The system is an Android app that loads a remote HTTP URL into a WebView and renders the returned content in the app UI. The relevant components are the user, `MainActivity`, the `WebView/WebViewClient`, the untrusted network, and `www.example.com`. The key trust boundary is where WebView traffic leaves the app process and crosses the untrusted network. The protected assets are the rendered WebView content, the HTTP request/response data in transit, and the user's trust in app-delivered content.

The attacker is an on-path network adversary, such as a same-Wi-Fi attacker, rogue access point, or compromised router. Because the WebView uses HTTP, the attacker can observe requests and responses and modify response content before it is rendered inside the app. This supports content

Evidence and security meaning

`AndroidManifest.xml:13: INTERNET`. App can perform network traffic.

`AndroidManifest.xml:26: usesCleartextTraffic="true"`. App permits cleartext HTTP.

`AndroidManifest.xml:30-36: launcher MainActivity`. Vulnerable path is normally reachable.

`MainActivity.java:31-32: WebView and WebViewClient initialized`. WebView is the network-rendering component.

`MainActivity.java:38: loadUrl("http://www.example.com")`. App actively loads cleartext HTTP content.

`MainActivity.java:34-35: sslErrorHandler.proceed()`. Supporting evidence: TLS failures are not rejected.

`MainActivity.java:39-43: HostnameVerifier returns true`. Supporting evidence only; no binding to this HTTP path is required.

Table 1: Code/config evidence for insecure WebView transport.

tampering, including attacker-chosen or stale content in the WebView. Figure 1 shows the resulting attack surface.

3 Evidence and Proof of Concept

Table 1 gives the static evidence. The primary evidence is the combination of `usesCleartextTraffic="true"` and `loadUrl("http://www.example.com")` in a launcher-reachable activity. The SSL-error and hostname-verifier code are supporting evidence of unsafe transport posture, but the primary demonstrated issue remains HTTP cleartext WebView loading.

A reproducible static PoC is:

1. Decompile the APK and inspect the manifest and sources.
2. Confirm that the manifest declares `INTERNET` and sets `usesCleartextTraffic="true"`.
3. Confirm that the launcher activity reaches `MainActivity`.
4. Confirm that `MainActivity` attaches a `WebViewClient`

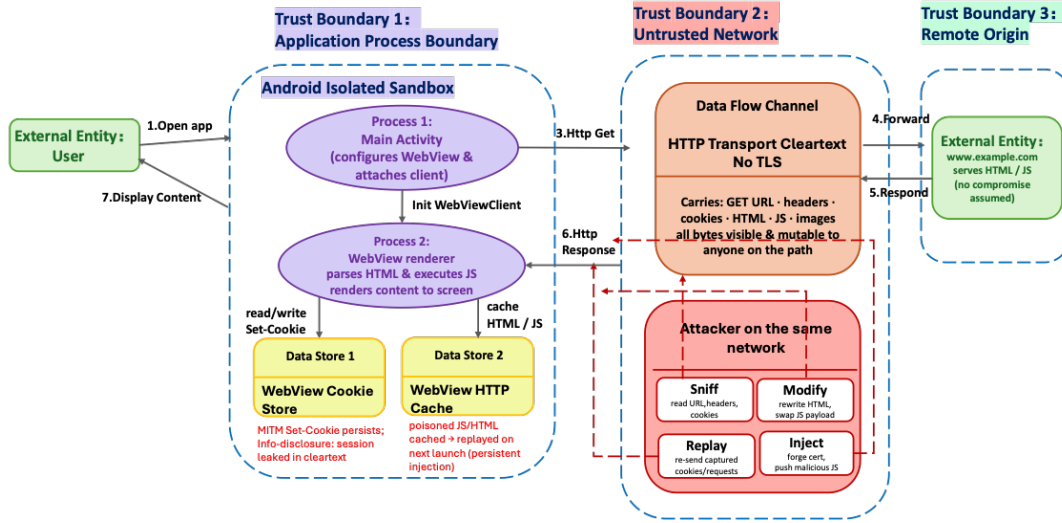


Figure 1: Network-focused model. The key trust boundary is between the app process and the untrusted network.

and calls `loadUrl("http://www.example.com")`.

This proves that normal app launch can create an HTTP WebView request. A safe dynamic validation, if performed in a lab emulator, is to route the emulator through a controlled proxy, launch the app, observe the HTTP request to `www.example.com`, replace the response body with benign marker HTML, and verify that the WebView renders the modified content. This dynamic step is not needed for the core static finding, but it demonstrates the practical MITM consequence without attacking a real third party.

4 Vulnerability and Impact

The vulnerability is insecure transport for WebView content. HTTP provides no channel confidentiality, integrity, or endpoint authentication. Therefore, any on-path attacker can read the request and response, modify the response body, and cause attacker-chosen content to appear inside the app's WebView. Because this content is rendered in the app UI rather than a separate browser context, users may treat it as app-delivered or app-endorsed content.

The realistic attack path is: the victim launches the app; `MainActivity` initializes the `WebView`; the `WebView` loads `http://www.example.com`; the request leaves over HTTP cleartext; an on-path attacker reads or modifies the response; and the `WebView` renders the tampered page inside the app. The strongest supported impact is `WebView` content integrity/authenticity compromise, HTTP request/response confidentiality and integrity loss, and possible misleading or phishing-style content injection inside the app interface. The evidence does not support claiming account takeover, backend compromise, or theft of app credentials.

5 Mitigation and Verification

The fix should remove the cleartext path and fail closed when transport validation fails:

1. Replace `http://www.example.com` with a valid `https://` endpoint.
2. Set `android:usesCleartextTraffic="false"`.
3. Add a restrictive Network Security Config that denies cleartext by default and only permits trusted HTTPS origins.
4. Replace `sslErrorHandler.proceed()` with `sslErrorHandler.cancel()`.
5. Remove permissive `HostnameVerifier` logic and rely on Android's default certificate and hostname validation unless a correct pinning design is required.

These changes address the root cause. HTTPS provides confidentiality, integrity, and endpoint authentication; disabling cleartext prevents HTTP regressions; and fail-closed TLS handling prevents invalid-certificate content from being rendered as trustworthy. Verification should check statically that no `loadUrl("http://...")`, `usesCleartextTraffic="true"`, or `sslErrorHandler.proceed()` remains, and dynamically that a proxy no longer sees HTTP traffic and invalid certificates are blocked.

6 Conclusion

`a2_case1.apk` permits cleartext transport and loads an HTTP URL inside a launcher-reachable `WebView`. This creates a credible MITM-style path where an on-path attacker can read or modify `WebView` traffic and cause tampered content to be rendered inside the app UI. Enforcing HTTPS, disabling cleartext, and rejecting TLS validation failures removes the demonstrated attack path.